Classical ML, done properly

The baseline every AI proposal has to beat, and how to measure it without fooling yourself.

60 min | Recorded | Class 3 of 15

3

What this session covers

01
What a model learns from
3 slides

02
Measuring it honestly
6 slides

03
Two more jobs classical ML does well
4 slides

04
Code walkthrough
7 slides

© 2026 Devansh Parapalli

Section 1 of 4

What a model learns from

Examples in, a pattern out.

01

A model learns a mapping from examples, so the examples decide what it can learn

02

The word for today

03

Always guessing the biggest queue gets 9%; rules get 60%; a simple model gets 90%

© 2026 Devansh Parapalli

A model learns a mapping from examples, so the examples decide what it can learn

Each old ticket is one example. The **features** are what the model gets to look at: the words, the system named, whether it's period end. The **label** is the right answer: the queue that fixed it.

Training finds weights that turn features into labels. If the old labels were sloppy, the model learns the sloppiness too. About 5% of ours are.

Features
"Harbor MDM enrolment stuck for CW-48213" · contingent · portal

Label
cw-device

© 2026 Devansh Parapalli

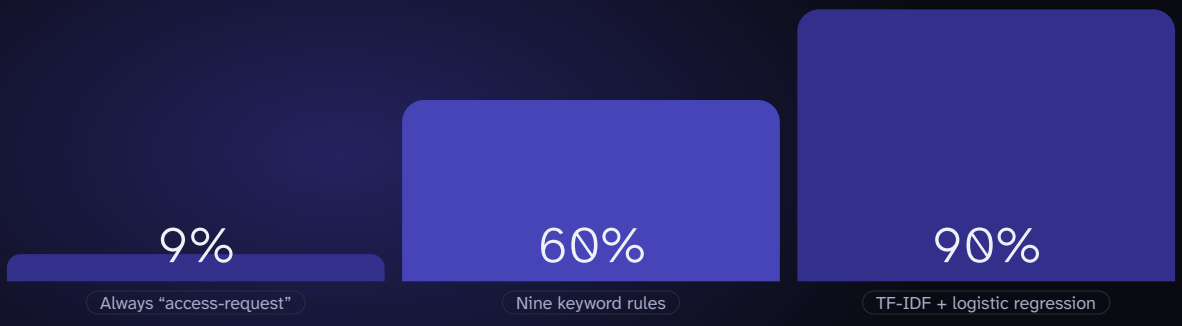
The word for today

If a model can't beat this, it doesn't ship

Baseline

© 2026 Devansh Parapalli

Always guessing the biggest queue gets 9%; rules get 60%; a simple model gets 90%



Source: Measured on the generated ticket set, 600 held-out tickets

© 2026 Devansh Parapalli

The rules are the ones a support lead would write on day one. They're in code/class03/classify.py. 80% from nine lines is a strong result; say so.

Section 2 of 4

Measuring it honestly

Most bad models look great on the data they were trained on.

01
Hold back a test set, like an exam you haven't seen

02
Leakage is when the answer sneaks into the training data, and the score lies

03
98.9% accuracy means nothing when almost no tickets are P1

04
Precision and recall ask two different questions

© 2026 Devansh Parapalli

Hold back a test set, like an exam you haven't seen

Split before you look at anything.

01 Train on most of it
80% of the tickets. The model sees these examples and their answers.

02 Test on the rest
20% it never saw. This is the only score that counts.

03 Split by time if time matters
Train on last year, test on this quarter. That's how it'll be used.

© 2026 Devansh Parapalli

Leakage is when the answer sneaks into the training data, and the score lies

- The description says “moved to the invoice queue”
- Resolution notes, written after the fact, are used as input
- The same ticket appears twice, once on each side of the split

Each one makes the test score look excellent and the live results poor.

Rule of thumb: only use what was known at the moment the prediction would be made.

© 2026 Devansh Parapalli

98.9% accuracy means nothing when almost no tickets are P1

Out of 3000 tickets, 33 are P1, nearly all payroll mismatches. A model that always says “not P1” is right 98.9% of the time.

It has never caught a single P1.

tickets	3000
actually P1	33
accuracy of “never P1”	98.9%
P1s caught	0

© 2026 Devansh Parapalli

Precision and recall ask two different questions

Precision	Of the tickets it sent to payroll, how many really were payroll? Low precision wastes analysts' time.
Recall	Of the real payroll mismatches, how many did it send there? Low recall means an RCA starts late.
F1	One number that balances the two. Useful for comparing; hides which one is weak.
Confusion matrix	The four counts behind all of it: hits, misses, false alarms, correct passes.

© 2026 Devansh Parapalli

The confusion matrix is four counts, and every metric comes from them

	PREDICTED PAYROLL	PREDICTED OTHER
Actually payroll	23 · caught	2 · missed
Actually other	3 · false alarm	572 · correct pass

Source: Test set, payroll-mismatch against everything else

© 2026 Devansh Parapalli

Which one to favour depends on what a mistake costs

Favour recall

- Payroll mismatches and P1s
- Odd readings on the integration hosts
- A miss costs more than a false alarm

Favour precision

- Auto-replying to InvoiceHub status checks
- Auto-extending CW Portal accounts
- A wrong action costs more than a miss

© 2026 Devansh Parapalli

Section 3 of 4

Two more jobs classical ML does well

Finding the odd one out, and saying what comes next.

01

Anomaly detection found every injected fault in the host signal feed, with no labels

02

Ticket volume doubles around period close, so a forecast has to know the calendar

03

Classical ML beats an LLM when the input is structured and the volume is high

04

Building a classical model always follows the same six steps

© 2026 Devansh Parapalli

Anomaly detection found every injected fault in the host signal feed, with no labels

Our monitoring sends hourly CPU, memory and job latency for 60 integration hosts. An isolation forest keeps splitting the data at random; readings that get separated in very few splits are the odd ones.

Nobody had to write a threshold for each host.

readings flagged	101
injected faults caught	52 / 52

© 2026 Devansh Parapalli

About half the flags are real faults, half are busy hours that look unusual. That’s the precision trade-off again. Show code/class03/signals.py.

Ticket volume doubles around period close, so a forecast has to know the calendar

Ledgerline, payroll and S.C. billing tickets pile up in the last two days of the month and the first three of the next. A seasonal model learns the weekly shape and the close spike.

The baseline to beat is simple: this close looks like last close.

tickets a normal weekday	5.4
tickets a close weekday	12.2

© 2026 Devansh Parapalli

Classical ML beats an LLM when the input is structured and the volume is high

Classical ML

- Fractions of a cent per million predictions
- Milliseconds, runs on a laptop CPU
- Same input, same answer, every time
- Weights you can inspect

LLM

- Cost per call, every call
- Hundreds of milliseconds to seconds
- Answers vary between runs
- Hard to explain a single decision

© 2026 Devansh Parapalli

Building a classical model always follows the same six steps

1	2	3	4	5	6
Collect labelled history	Split before looking	Score the baseline	Train a simple model	Measure on the test set	Decide: ship, improve, or stop

© 2026 Devansh Parapalli

Section 4 of 4

Code walkthrough

A ticket classifier in scikit-learn, scored against the baseline and the rules.

01

Split first, and keep the queue mix the same on both sides

02

The rules a support lead would write on day one are nine regular expressions

03

The baseline and the real model are each one line

04

Read the report from the worst queue up, not from the average down

© 2026 Devansh Parapalli

Split first, and keep the queue mix the same on both sides

code/class03/classify.py

```
df = tickets()
train, test = train_test_split(df, test_size=0.2, stratify=df.queue, random_state=7)
```


© 2026 Devansh Parapalli

The rules a support lead would write on day one are nine regular expressions

code/class03/classify.py

```
RULES = [
    (r"password|locked out|account locked", "password-reset"),
    (r"invoicehub|invoice import|batch ih-", "invoice-import-status"),
    (r"reopen|closed too early|po-4500", "po-reopen"),
    (r"harbor|enrol|lol image", "cw-device"),
    (r"payroll|paygrid", "payroll-mismatch"),
    # ... four more
]
```


© 2026 Devansh Parapalli

The baseline and the real model are each one line

code/class03/classify.py

```
baseline = DummyClassifier(strategy="most_frequent").fit(train.text, train.queue)
model = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2),
                      LogisticRegression(max_iter=1000)).fit(train.text, train.queue)
print(classification_report(test.queue, model.predict(test.text)))
```


© 2026 Devansh Parapalli

Read the report from the worst queue up, not from the average down

- The weakest queue is **password-reset**, with recall 0.76 on 42 test tickets
- Those are the vague ones: “Account locked” with no system named, so it looks like an access request
- Compare the overall score to the baseline before saying anything is good

© 2026 Devansh Parapalli

Show the report live. Point at payroll first, then the average.

On tickets written the way people actually write, it drops to 71%, and it never says “I don’t know”

We wrote 300 tickets by hand, across the whole company: one-word titles, typos, two problems in one, forwarded chains. The model trained on the generated set gets 71% of them. The day-one rules get 53%.

The five that belong in no queue, a test ticket, a phishing report, a suspicious bank-change email, it puts in one anyway.

“Vendor bank change request” (a fraud attempt) went to invoice-import

“Break room fridge” went to ledger-close

“Payroll - Leeds double payment” went to customer-payments

© 2026 Devansh Parapalli

Run code/class03/handwritten.py live. The fix for “not ours” tickets is a confidence threshold plus a person, not a better model. The bank-change one matters: it should have gone to security, and a model that routes it to invoices makes fraud easier.

Where today sits on the card

1. A script	The steps are known and don't change.
2. A rule	The decision is known and must be explainable line by line.
3. Classical ML	**Labelled history, and the input is a table or short text.**
4. A small trained model	Lots of text, images or audio, enough labels, cost per call matters.
5. An LLM with a prompt	Varied language in and out, few labels, someone checks the output.
6. An LLM plus retrieval, tools or agents	Needs private knowledge, has to act, or takes several steps.
7. Not yet	No ground truth, needs a guarantee, or nobody owns the result.

© 2026 Devansh Parapalli

What to do next

1. Run `code/class03/classify.py` and find the weakest queue on your machine.
2. Change the split to time-based and see what happens to the score.
3. Read the class 4 pre-reading on regular expressions.

Questions after class: the course channel.

© 2026 Devansh Parapalli